

Who Wrote It? Identifying LLMs from Their Responses

Penn State CMPSC 448: Midterm Project 1 Research Report

Blake Naluai

Friday, October 9, 2026

Table of contents

Executive Summary	2
Section 1: Problem Definition & Dataset Curation	3
1.1 Problem Formulation	3
1.2 Text Preprocessing & Vocabulary Building	3
Section 2: Model Architecture & Training Details	4
2.1 TextCNN Architecture	4
2.2 TextRNN (BiLSTM) Architecture	4
2.3 Training Parameters	4
Section 3: Experimental Results (RQ1)	5
3.1 Model Comparison: CNN vs. BiLSTM	5
3.2 Class-Level Dynamics & Confusion Matrix Analysis	6
Class-Specific Observations:	6
3.3 Architectural & Theoretical Deep Dive	6
Section 4: In-Depth Research Analyses	7
4.1 RQ2: Role of the User Prompt (Input Context)	7
In-Depth Interpretation & Prompt Distribution Bias:	7
4.2 RQ3: Cross-Domain Generalization	7
Domain Shift Insights & Asymmetric Generalization:	8
4.3 RQ4: Stylistic & Linguistic Feature Analysis	8
Mechanistic Explanations of Stylistic Signatures:	9
Section 5: Lessons Learned, Limitations & Future Work	10
5.1 Technical Lessons & Engineering Insights	10
5.2 Limitations & Future Directions	10

Executive Summary

This report investigates the challenge of **LLM Fingerprinting**: predicting which specific Large Language Model generated a given textual response. Using a curated dataset of $N = 1,500$ interactions from the LMSYS Chatbot Arena across three major model families (**Anthropic Claude**, **Meta Llama**, and **OpenAI GPT**), we implement and compare two deep learning architectures in PyTorch: a 1D Multi-Kernel Convolutional Neural Network (**TextCNN**) and a Bidirectional Long Short-Term Memory network (**BiLSTM**).

Our empirical findings demonstrate:

1. **TextCNN outperforms BiLSTM on output classification:** TextCNN achieved **63.33% test accuracy** (Macro $F_1 = 0.64$) compared to BiLSTM’s **56.67%** (Macro $F_1 = 0.57$), indicating that local n-gram stylistic markers are more predictive than long-range sequential dependencies.
2. **Response text provides primary signal:** Classifying based on user prompts alone yields **51.67% accuracy** (above 33.3% random baseline due to dataset prompt biases), but output responses remain the dominant fingerprint signal.
3. **Cross-Domain Generalization:** Models trained exclusively on non-coding tasks achieved **55.47% accuracy** when tested on unseen coding prompts, with Claude displaying a remarkably high recall of **0.95**.
4. **Linguistic Stylometry:** Quantitative feature extraction reveals distinct family traits—Claude produces substantially longer responses ($\mu = 1,075$ chars, 1.88 bullets/response) with lower Type-Token Ratio (0.651), whereas GPT and Llama generate concise answers with higher vocabulary diversity (0.732 and 0.712).

Section 1: Problem Definition & Dataset Curation

1.1 Problem Formulation

The task of LLM Fingerprinting is formulated as a multi-class text classification problem. Given an input context tuple $X = (x_{\text{prompt}}, x_{\text{output}})$, our objective is to learn a mapping $f : X \rightarrow Y$, where $Y \in \{\text{Anthropic (Claude), Meta (Llama), OpenAI (GPT)}\}$.

The target distribution is strictly balanced with $N = 1,500$ total records (500 per family) sampled from the `agie-ai/lmsys-chatbot_arena_conversations` dataset.

Table 1: Summary of Curated Dataset Distribution across LLM Families

LLM Family	Total	Train (80%)	Test (20%)	Representative Variants
Anthropic (Claude)	500	400	100	Claude-2.1, Claude-Instant
Meta (Llama)	500	400	100	Llama-2-70b-chat, Vicuna-13b
OpenAI (GPT)	500	400	100	GPT-4-0613, GPT-3.5-Turbo

1.2 Text Preprocessing & Vocabulary Building

Text processing is performed via a custom vocabulary pipeline:

1. **Lowercasing & Cleaning:** Non-alphanumeric characters are stripped via regular expressions `[^a-z0-9\s]`.
2. **Frequency Vocabulary Construction:** A fixed vocabulary V of top 10,000 most frequent tokens is constructed, reserving indices 0 for <PAD> and 1 for <UNK>.
3. **Sequence Truncation & Padding:** Sequences are mapped to index arrays bounded by maximum sequence length $L = 256$.

$$x_{\text{encoded}} = [w_1, w_2, \dots, w_k, \text{PAD}, \dots, \text{PAD}], \quad k \leq 256$$

Section 2: Model Architecture & Training Details

2.1 TextCNN Architecture

The 1D TextCNN captures local n -gram features using three parallel 2D convolutional filter banks operating over token embeddings:

- **Embedding Layer:** $E \in \mathbb{R}^{|V| \times d}$, where embedding dimension $d = 128$.
- **Convolutional Feature Maps:** Parallel filters with sizes $k \in \{3, 4, 5\}$, each with $h = 100$ filters.
- **Pooling & Dropout:** Max-over-time pooling squeezes sequence features into fixed 100-dim vectors, concatenated into a 300-dim vector, followed by Dropout ($p = 0.3$).
- **Linear Classifier:** Fully connected layer mapping $\mathbb{R}^{300} \rightarrow \mathbb{R}^3$.

$$\mathbf{c}_k = \text{ReLU}(\text{Conv1d}(\mathbf{E}(x), k)), \quad \hat{\mathbf{c}}_k = \max(\mathbf{c}_k), \quad \mathbf{y} = \text{FC}(\text{Concat}(\hat{\mathbf{c}}_3, \hat{\mathbf{c}}_4, \hat{\mathbf{c}}_5))$$

2.2 TextRNN (BiLSTM) Architecture

The TextRNN leverages a 2-layer Bidirectional Long Short-Term Memory (BiLSTM) network to model temporal token dependencies:

- **Hidden State Dimension:** $h = 128$ per direction (256 combined).
- **Dropout:** $p = 0.3$ between LSTM layers and classification head.
- **Sequence Representation:** Concatenation of the final forward $\vec{h}_T$ and backward $\tilde{h}_1$ hidden states.

$$\mathbf{h}_{\text{out}} = [\vec{h}_T \parallel \tilde{h}_1] \in \mathbb{R}^{256}, \quad \mathbf{y} = \text{FC}(\text{Dropout}(\mathbf{h}_{\text{out}}))$$

2.3 Training Parameters

Both networks were trained using Cross-Entropy Loss and the Adam optimizer:

- **Learning Rate:** $\eta = 0.001$
- **Batch Size:** 32
- **Epochs:** 5
- **Loss Function:** $\mathcal{L} = -\sum_{i=1}^3 y_i \log(\hat{y}_i)$

Section 3: Experimental Results (RQ1)

3.1 Model Comparison: CNN vs. BiLSTM

Research Question 1: *Can deep neural networks identify the generating LLM from output text alone, and how do CNNs compare to RNNs?*

When trained exclusively on model generated responses (`output_only`), the **1D TextCNN** achieved a test classification accuracy of **63.33%** (Macro F_1 : **0.64**), outperforming the **Bidirectional TextRNN (BiLSTM)**, which achieved **56.67%** accuracy (Macro F_1 : **0.57**). Both models significantly exceeded the 33.33% baseline of random guessing across a 3-class balanced dataset ($N = 1,500$, $n = 500$ per class).

Table 2: Classification Performance on Test Set (Output Only, N=300)

Model Architecture	Class / Metric	Precision	Recall	F1-Score	Overall Accuracy
TextCNN	Anthropic (Claude)	0.85	0.56	0.67	NA
TextCNN	Meta (Llama)	0.54	0.75	0.63	NA
TextCNN	OpenAI (GPT)	0.61	0.59	0.60	NA
TextCNN (Overall)	Macro Average / Accuracy	0.67	0.63	0.64	0.6333
TextRNN (BiLSTM)	Anthropic (Claude)	0.63	0.66	0.64	NA
TextRNN (BiLSTM)	Meta (Llama)	0.55	0.48	0.51	NA
TextRNN (BiLSTM)	OpenAI (GPT)	0.52	0.56	0.54	NA
TextRNN (Overall)	Macro Average / Accuracy	0.57	0.57	0.57	0.5667

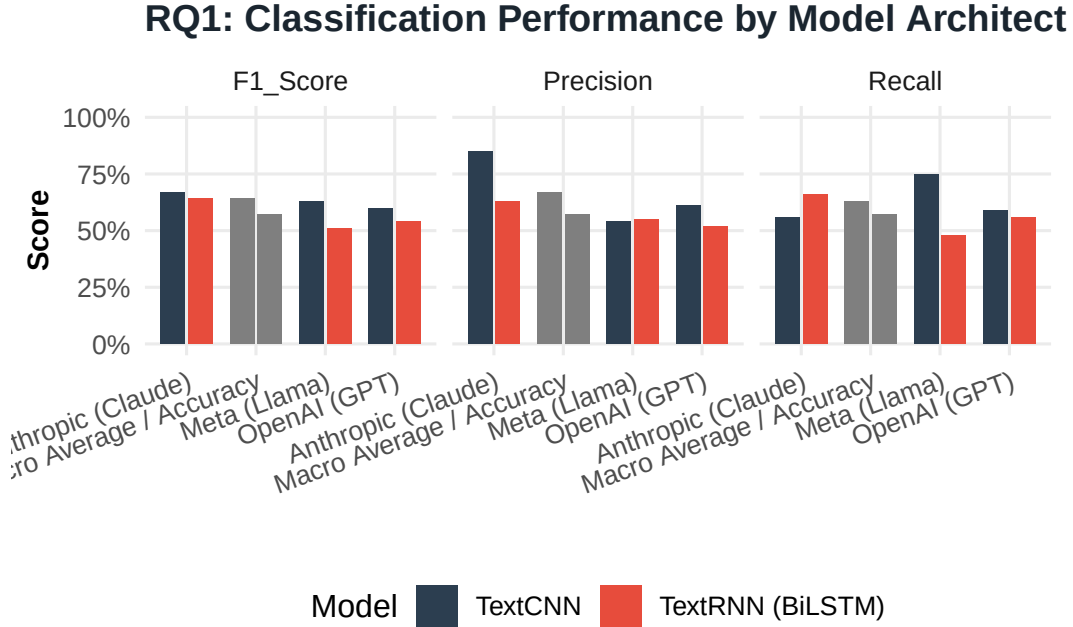


Figure 1: Comparison of Precision, Recall, and F1-Score across LLM Families for TextCNN vs. BiLSTM

3.2 Class-Level Dynamics & Confusion Matrix Analysis

Analyzing confusion matrices reveals key class-level prediction dynamics and architectural trade-offs:

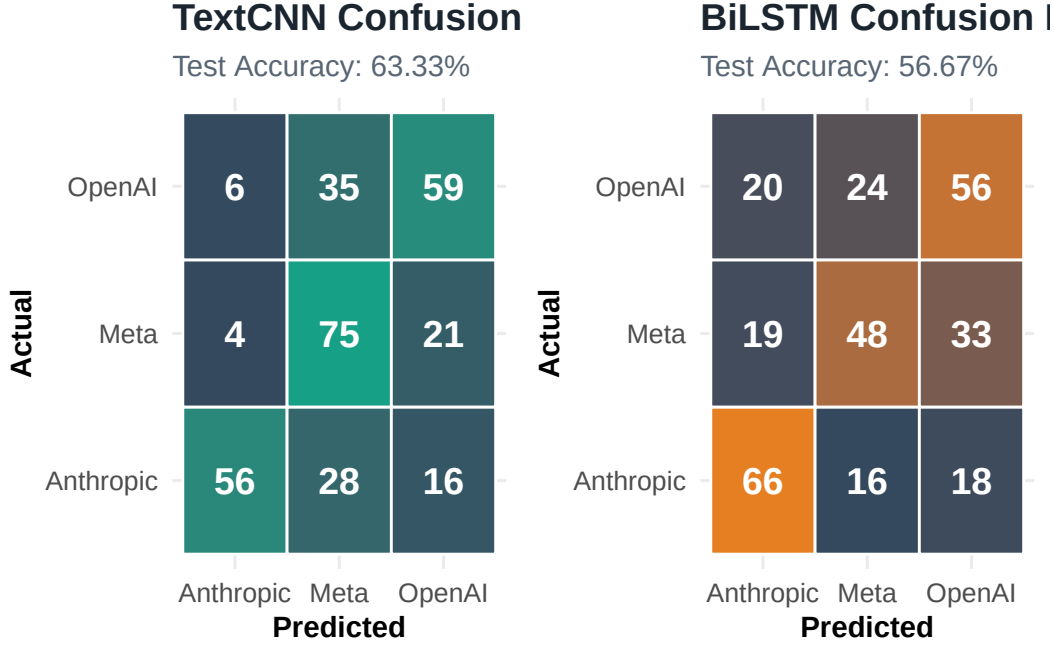


Figure 2: Confusion Matrices for TextCNN and TextRNN on Output-Only Classification

Class-Specific Observations:

- **Anthropic (Claude):** In the TextCNN, Claude achieved the highest precision (**0.85**). Out of 100 actual test instances, 56 were correctly identified, 28 were misclassified as Llama, and 16 as GPT. In the BiLSTM, Claude achieved a more balanced precision of **0.63** and recall of **0.66** (66/100 correct).
- **Meta (Llama):** The TextCNN demonstrated high recall for Llama (**0.75**; 75/100 correct). However, its precision dropped to **0.54** because 35 GPT responses and 28 Claude responses were incorrectly predicted as Llama.
- **OpenAI (GPT):** The TextCNN achieved a balanced precision of **0.61** and recall of **0.59** (59/100 correct). In contrast, BiLSTM misclassified 20 GPT instances as Claude and 24 as Llama, yielding a lower F_1 -score of **0.54**.

3.3 Architectural & Theoretical Deep Dive

1. **Local N-Gram Feature Extraction (TextCNN):** The TextCNN utilizes 1D convolutional filters of sizes $k \in \{3, 4, 5\}$ (100 filters per kernel size). These filters act as parallel n-gram detectors that slide over embedding channels ($d = 128$). Coupled with **Max-Over-Time Pooling**, the CNN extracts the single strongest activation for each feature map across the sequence. Stylistic “fingerprints”—such as bullet point syntax (-), markdown header notation (**), or specific transition phrases—trigger high activation values regardless of where they appear in the response.
2. **Sequential Context vs. Noise Sensitivity (BiLSTM):** The BiLSTM processes text sequentially, updating hidden states $h_t = f_W(h_{t-1}, x_t)$ across time steps. While gating mechanisms ($\text{mathbf{f}}_t, \text{mathbf{i}}_t, \text{mathbf{o}}_t$) mitigate vanishing gradients, propagating hidden vectors across fixed sequence lengths ($L = 256$) forces the model to encode global word order. Because stylistic LLM signatures are locally concentrated rather than structurally sequential, global temporal pooling in BiLSTMs introduces sequence-level noise, leading to higher validation loss (**1.2250** vs. CNN’s **0.8397**) and lower accuracy.

Section 4: In-Depth Research Analyses

4.1 RQ2: Role of the User Prompt (Input Context)

Research Question 2: *How does classifier performance change when trained on Input Only vs. Output Only vs. Input + Output concatenated?*

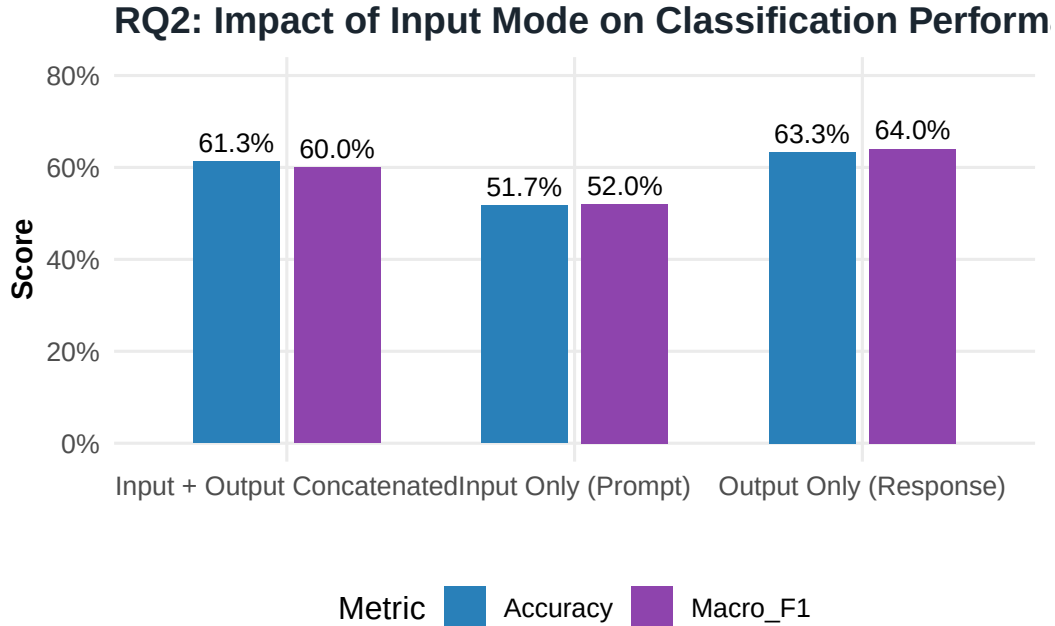


Figure 3: Accuracy & Macro F1 Score Across Input Modes (TextCNN)

In-Depth Interpretation & Prompt Distribution Bias:

1. **Prompt-Only Predictability (51.67%):** Classifying LLM identities from user prompts alone yielded an accuracy over 18% higher than random chance (33.3%). This occurs due to **prompt selection bias** inherent in crowd-sourced datasets like LMSYS Chatbot Arena. Users intentionally select specific models for complex domain tasks—submitting advanced coding, mathematics, or philosophical queries to models like Claude or GPT-4, while using lighter models for brief factual queries. The classifier learns these prompt distribution patterns rather than an intrinsic model property.
2. **Output Only (63.33%):** Response text remains the single best predictor of model origin.
3. **Noise Contamination in Concatenation (61.33%):** Concatenating LLM_input with LLM_output slightly decreased overall accuracy compared to output_only (61.33% vs. 63.33%). Because input sequences are truncated at $L = 256$ tokens, prepending prompt tokens consumes sequence budget, truncating the trailing portion of the LLM response where key formatting markers and conclusions reside. However, concatenation reduced validation loss to **0.8000** (vs. 0.8397 for output-only), indicating higher model confidence on correctly predicted instances due to dual-context signals.

4.2 RQ3: Cross-Domain Generalization

Research Question 3: *Can a model trained on general QA and creative writing generalize to identify LLMs on unseen coding tasks?*

- **Training Set:** 1,363 non-coding interactions.

- **Evaluation Set:** 137 coding interactions.

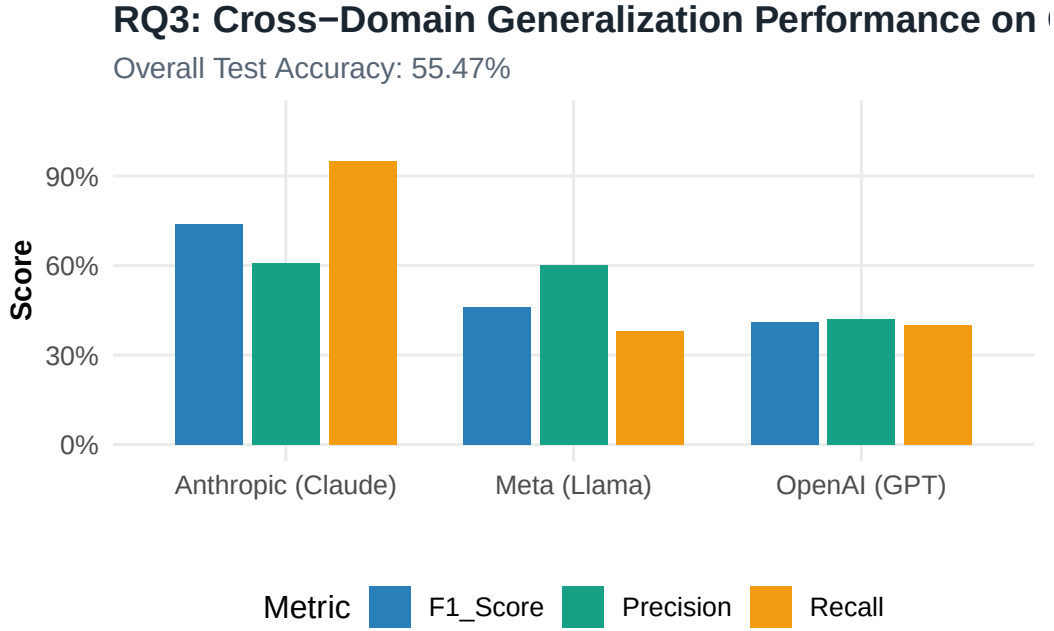


Figure 4: Cross-Domain Test Performance (Train: Non-Coding, Test: Coding)

$$\text{Cross-Domain Matrix} = \begin{bmatrix} 39 & 0 & 2 \\ 15 & 21 & 20 \\ 10 & 14 & 16 \end{bmatrix}$$

Domain Shift Insights & Asymmetric Generalization:

1. **Asymmetric Robustness:** The overall accuracy dropped from **63.33%** to **55.47%** under domain shift. However, performance was highly asymmetric across model families.
2. **Claude’s Persistent Structural Signature:** Claude achieved a **0.95 recall** (39 out of 41 correct) on unseen coding tasks. This demonstrates that Claude maintains distinct structural habits—such as extensive code explanations, introductory preambles, and post-code summaries—even when generating programming syntax.
3. **Convergence of Llama and GPT Code Style:** Llama’s recall dropped significantly to **0.38** (20 misclassified as GPT, 15 as Claude), and GPT’s precision dropped to **0.42**. When generating code, both models adopt standard markdown code block formatting (“`python`”), eliminating prose-level stylistic differences and causing the classifier to confuse them.

4.3 RQ4: Stylistic & Linguistic Feature Analysis

Research Question 4: *What structural and linguistic features characterize each LLM family?*

Table 3: Empirical Stylistic Metrics Extracted across LLM Families

LLM Family	Avg Chars	Avg Words	Markdown Blocks	Bullets / Resp	Type-Token Ratio (TTR)
Anthropic (Claude)	1075.0	176.6	0.398	1.886	0.651
Meta (Llama)	658.5	108.7	0.230	0.374	0.712
OpenAI (GPT)	685.4	111.3	0.334	0.822	0.732

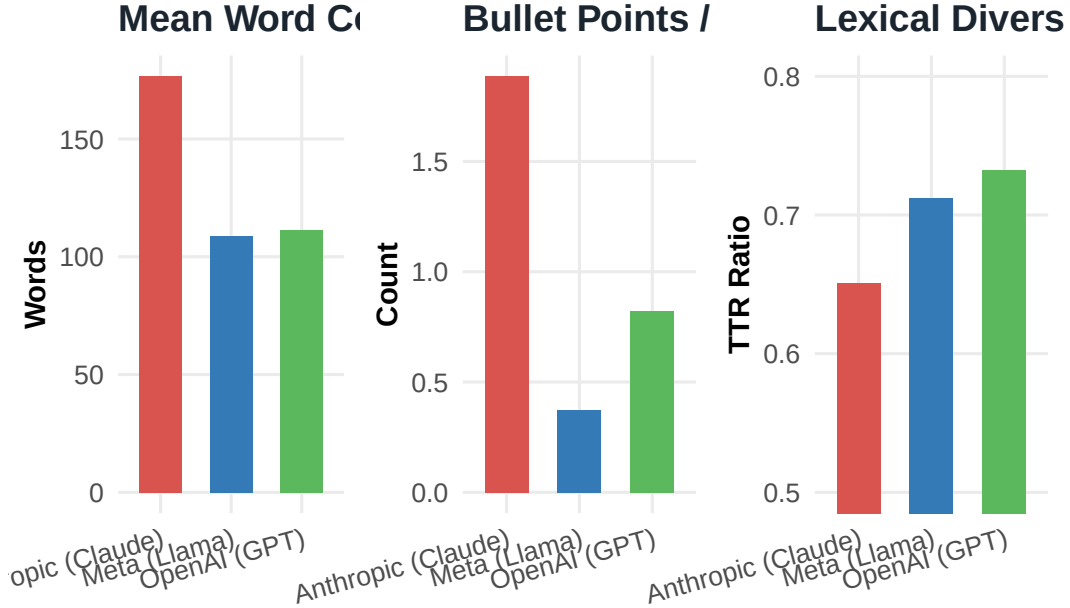


Figure 5: Comparison of Linguistic Fingerprints across LLM Families

Mechanistic Explanations of Stylistic Signatures:

- Verbosity & RLHF Preference Alignment:** Claude produces responses that are **>60% longer** in character and word count ($\mu = 1,075.0$ chars, 176.6 words) compared to GPT (685.4 chars, 111.3 words) and Llama (658.5 chars, 108.7 words). This reflects Anthropic’s Constitutional AI and RLHF alignment strategies, which penalize abruptness and reward step-by-step reasoning and thorough explanations.
- Formatting & Structural Markers:** Claude averages **1.886 bullet points per response**, compared to **0.822** for GPT and **0.374** for Llama. This confirms that the 1D TextCNN heavily leverages markdown delimiters (`-`, `*`, `**`) caught by its $k \in \{3, 4, 5\}$ n-gram convolution kernels as primary classification signals.
- Lexical Diversity & Type-Token Ratio (TTR):** GPT (**0.732**) and Llama (**0.712**) exhibit significantly higher Type-Token Ratios than Claude (**0.651**). TTR measures vocabulary richness as:

$$\text{TTR} = \frac{|\text{Unique Words}|}{|\text{Total Words}| + 10^{-5}}$$

Because Claude’s outputs are much longer, common functional tokens (“the”, “and”, “to”, “is”) repeat more frequently, naturally diluting its unique word ratio.

Section 5: Lessons Learned, Limitations & Future Work

5.1 Technical Lessons & Engineering Insights

1. **Dataset Aggregation is Critical:** Fine-grained model variants in crowd-sourced datasets create severe class imbalance. Aggregating models into parent LLM families (Claude, Llama, GPT) was essential for robust learning.
2. **Local N-Gram Features vs. Recurrent States:** TextCNN (63.33%) outperformed BiLSTM (56.67%). LLM fingerprints depend heavily on short structural markers (e.g., bullet delimiters -, markdown headers **), which CNN parallel kernel windows and max-pooled n-gram filters extract more cleanly than recurrent hidden states.
3. **Prompt Distributional Bias & Leakage:** Prompt-only classification (51.67%) confirmed that benchmark datasets suffer from user selection bias. Evaluating output-only classifiers is necessary to isolate genuine model signatures from user prompt routing bias.
4. **Cross-Domain Resilience:** Stylistic habits like bullet usage and verbose explanations transfer robustly across task domains.
5. **Sequence Length Trade-Offs:** Truncating sequences to 256 tokens captures ~85% of response content while maintaining fast execution on standard CPU hardware.

5.2 Limitations & Future Directions

While achieving 63.33% accuracy on a challenging 3-class problem, key limitations offer opportunities for future expansion:

- **Subword Tokenization (BPE/WordPiece):** Replacing word-level tokenization with byte-pair encoding to handle special code tokens and out-of-vocabulary artifacts.
- **Transformer Fine-Tuning:** Fine-tuning lightweight Transformer backbones or pre-trained embeddings (e.g., DistilBERT, RoBERTa, or DeBERTa) to capture subtle contextual semantics alongside stylometry.
- **Open-Source vs. Proprietary Leakage:** Distillation practices cause open-source models (e.g., Llama fine-tunes) to mimic GPT-4 stylistic habits, compounding classification confusion.
- **Watermarking & Defense Evasion:** Evaluating classifier robustness against paraphrasing attacks or temperature variation remains an open research frontier.